

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #7 - Semana 8 – Sivarama - Capitulo 12
Más sobre procedimientos

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 09/09/2024

Bibliografía:

[1] S. P. Dandamudi, *Guide to Assembly Language Programming in Linux*. Springer Science & Business Media, 2005, pp. 255–272.

Abstract—*This document covers essential assembly language concepts: managing local variables using registers or the stack, modular program structure for easier assembly and error reduction, handling procedures with variable parameters via the stack, and a Fibonacci calculation example using stack-based local variables. It also explains the GLOBAL and EXTERN directives for module integration and provides notes on Linux memory management and stack cleanup practices.*

I. Manejo de variables locales

En el lenguaje ensamblador, las variables locales de un procedimiento se pueden manejar utilizando registros o el stack. Los registros ofrecen eficiencia pero son limitados en número y no son adecuados para procedimientos no terminales o recursivos. Por otro lado, el uso del stack permite la creación de un marco de pila para las variables locales, garantizando que desaparezcan al finalizar el procedimiento. Por ejemplo, en el procedimiento compute de C, las variables locales se almacenan en el stack y se accede a ellas mediante el puntero de base (EBP).

II. Programas con múltiples módulos

Los programas grandes se dividen en módulos para facilitar su manejo. Cada módulo se guarda en un archivo separado. Los beneficios de esta técnica incluyen la posibilidad de ensamblar solo los módulos modificados, simplificar las modificaciones y reducir el riesgo de errores en archivos extensos. Para ensamblar programas de múltiples módulos, se utilizan directivas como GLOBAL y EXTERN para definir la visibilidad de las etiquetas y procedimientos entre módulos.

III. Procedimientos con número variable de parámetros

Los procedimientos que aceptan un número variable de argumentos utilizan el stack para manejar los parámetros. El último argumento pasado al procedimiento indica la cantidad de parámetros. Este método permite pasar un número indefinido de argumentos, limitados solo por el tamaño del stack. En el ejemplo proporcionado, el procedimiento `variable_sum` suma una cantidad variable de enteros recibidos en el stack y devuelve el resultado.

IV. Ejemplo de cálculo de números de Fibonacci

El código de ejemplo calcula el mayor número de Fibonacci menor o igual a un número dado. Utiliza dos variables locales en el stack para almacenar los dos números de Fibonacci más recientes. El procedimiento `fibonacci` realiza cálculos sucesivos hasta que el número de Fibonacci excede el valor dado. Se muestra cómo las variables locales se manejan en el stack y cómo el espacio de la variable local se limpia al finalizar el procedimiento.

V. Directivas GLOBAL y EXTERN

La directiva `GLOBAL` hace que las etiquetas sean accesibles desde otros módulos, permitiendo que los procedimientos y variables sean utilizados por diferentes partes del programa. La directiva `EXTERN` indica al ensamblador que ciertas etiquetas se encuentran en otros módulos, dejando "espacios" en el archivo objeto para que el enlazador los complete más tarde. Estas directivas facilitan el ensamblaje y la integración de programas modulares.

VI. Notas adicionales

En sistemas Linux, el prefijo de segmento no es necesario debido a que la memoria se trata como un solo segmento físico dividido en segmentos lógicos. La memoria se organiza en segmentos de código y datos, y el espacio de pila está separado del espacio del núcleo. Se recomienda utilizar un registro de índice para representar el desplazamiento relativo al EBP para un acceso más flexible a los parámetros en el stack. Además, el espacio de parámetros en el stack debe ser limpiado adecuadamente por el procedimiento principal, multiplicando el conteo de parámetros por el tamaño de cada parámetro y ajustando el registro ESP.